

Relazione Tecnica di Progetto

Wiki Collaborativo per Appunti

Domenico Aloï

Anno Accademico 2025/26 - Ingegneria del Software

Indice

1	Introduzione	4
1.1	As-Is	4
1.2	To-Be	4
2	Requisiti	6
2.1	Requisiti Funzionali (RF)	6
2.2	Requisiti Non Funzionali (RNF)	7
3	Glossario	9
4	Diagrammi dei casi d'uso	11
4.1	Dettagli del Caso d'Uso UC1	12
4.2	Dettagli del Caso d'Uso UC2	15
4.3	Dettagli del Caso d'Uso UC3	19
4.4	Matrice di tracciabilità	21
5	Progettazione	22
5.1	Architettura Logica (Sottosistemi)	22
5.2	Architettura Fisica (Deployment)	23
5.3	Processo di Sviluppo e Gestione	24
5.3.1	Pianificazione delle Attività (Diagramma di GANTT) .	24
5.3.2	Metodologia di Sviluppo	24
5.3.3	Approccio allo Sviluppo	25
5.4	Diagrammi delle Classi	25
5.4.1	Diagramma delle Classi FE	26
5.4.2	Diagramma delle classi BE Sottosistema: Bootstrap & Orchestration	26
5.4.3	Diagramma delle classi BE Sottosistema: Persistence & Security	27
5.4.4	Diagramma delle classi BE Sottosistema: Strategy & Query Object	28

5.4.5	Diagramma delle classi BE Sottosistema: Gestione Stato e Cronologia	29
5.5	Diagrammi di Sequenza	29
5.5.1	Diagramma di sequenza: RF6 (Creazione Appunto) . . .	30
5.5.2	Diagramma di sequenza: RF8 (parte Salvataggio Nuova Versione)	30
5.5.3	Diagramma di sequenza: RF9 (Ricerca Appunti)	31
5.6	Diagrammi di Attività	32
5.6.1	Diagramma delle Attività: Login (RF2)	32
5.6.2	Diagramma delle Attività: Visualizza Appunto (RF5) .	33
5.7	Progettazione dei Dati	34
5.7.1	Organizzazione del File System	35
5.8	Specifica REST API	36
5.8.1	Endpoint	37
6	Architettura	45
6.1	Design Pattern Model-View-Presenter (MVP)	45
6.1.1	Mapping nel codice	46
6.2	Design Pattern Table Data Gateway	46
6.2.1	Mapping nel codice	47
6.3	Design Pattern Strategy	47
6.3.1	Mapping nel codice	48
6.4	Design Pattern Memento	49
6.4.1	Mapping nel codice	49
6.5	Design Pattern Protection Proxy	50
6.5.1	Mapping nel codice	51
7	Piano di Test	53
7.1	Introduzione alla Validazione	53
7.2	Casi di Test e Tracciabilità	53
7.3	Considerazioni sulla Robustezza	55
8	Manuale Tecnico	56
8.1	Introduzione	56
8.2	Requisiti di Sistema	56
8.3	Inizializzazione del Database	56
8.3.1	Logica di Popolamento	57
8.4	Script di Automazione	57
8.4.1	Avvio del Progetto (<code>run_project.sh</code>)	57
8.4.2	Arresto del Progetto (<code>stop_project.sh</code>)	57
8.4.3	Reset Totale (<code>reset_all.sh</code>)	57

8.5	Procedura di Installazione Standard	58
8.6	Credenziali di Test	58
8.7	Risoluzione dei Problemi	58

Capitolo 1

Introduzione

La richiesta consiste nel creare un sistema dove gli studenti possono caricare, modificare e versionare appunti per i corsi. Principalmente si tratterà di un editor di testo (semplice markdown o text-area) dotato di cronologia delle versioni (semplificata) e un'organizzazione per corso/argomento.

1.1 As-Is

L'attuale gestione e condivisione degli appunti tra gli studenti universitari avviene prevalentemente in modo frammentato e manuale. Gli appunti sono dispersi su diverse piattaforme (email, chat, cloud personali), rendendo difficile la reperibilità del materiale. Non avendo un sistema centralizzato è altamente probabile disperdere correzioni o aggiornamenti importanti. In questo contesto la collaborazione risulta molto complicata e limitata all'iniziativa personale.

1.2 To-Be

Il sistema **Wiki Collaborativo per Appunti** si propone di risolvere tali problematiche attraverso un sistema centralizzato di collaborazione tra gli studenti. Esso infatti permetterà di:

- **Condividere il materiale:** Gli appunti sono condivisi in un unico sistema alla portata di tutti e a disposizione di tutti.
- **Collaborare:** Ogni utente, dopo la registrazione, può contribuire aggiungendo nuovi appunti o modificando gli esistenti.

- **Controllare il materiale:** Ogni modifica agli appunti genera automaticamente una nuova versione. Questo consente agli utenti di navigare nella cronologia e, in caso di errori rilevati, ripristinare la vecchia versione di un appunto.

Capitolo 2

Requisiti

2.1 Requisiti Funzionali (RF)

RF1 Registrazione utente

- L'utente può creare un account fornendo email e password.
- Non è richiesto un sistema di validazione dell'email.

RF2 Login/Logout

- L'utente può accedere al sistema tramite credenziali e terminare la sessione (logout).

RF3 Visualizzazione corsi

- Qualsiasi utente (autenticato o visitatore) può visualizzare l'elenco dei corsi disponibili.

RF4 Visualizzazione argomenti di un corso

- Ogni corso presenta una lista di argomenti.
- L'utente può navigare agevolmente tra i corsi e i relativi argomenti.

RF5 Visualizzazione degli appunti

- L'utente può visualizzare gli appunti associati a un argomento.
- Ogni appunto è leggibile; il contenuto è visualizzato in formato HTML renderizzato (da sorgente Markdown o testo semplice).

RF6 Creazione appunti

- Gli utenti autenticati possono creare nuovi appunti.
- Inserimento obbligatorio del titolo.

- Ogni appunto fa riferimento ad un solo corso ed un solo argomento.

RF7 Modifica appunti

- Gli utenti autenticati hanno il permesso di modificare gli appunti esistenti.
- Ogni operazione di modifica genera automaticamente una nuova versione dell'appunto.

RF8 Cronologia delle versioni

- Visualizzazione dell'elenco delle versioni precedenti per ogni appunto (inclusi timestamp e autore).
- Possibilità di consultare una versione specifica del passato.
- Funzionalità di ripristino: una versione precedente può essere promossa a versione attuale.

RF9 Ricerca

- Sistema di ricerca per individuare appunti tramite il titolo.

RF10 Gestione corsi, argomenti e appunti

- Gli utenti con privilegi di amministratore possono creare, modificare o eliminare corsi, argomenti.
- Gli appunti sono eliminabili solo dall'amministratore.

2.2 Requisiti Non Funzionali (RNF)

RNF1 Architettura

- L'applicazione deve essere sviluppata seguendo il pattern architetturale Model–View–Presenter (MVP).
- Frontend e Backend devono essere completamente disaccoppiati e comunicare esclusivamente tramite API.

RNF2 Tecnologie

- Frontend: utilizzo di tecnologie standard (HTML5, CSS3, JavaScript). È consentito l'uso di Bootstrap o jQuery.
- Backend: sviluppo in PHP puro (senza l'ausilio di framework come Laravel o Symfony).
- Interfaccia API: implementazione di servizi RESTful per le operazioni CRUD.

- Divieto di Server-Side Rendering (SSR): il backend non deve generare codice HTML.

RNF3 Indipendenza Frontend/Backend

- Il frontend deve essere progettato per poter interagire con backend alternativi (es. Java, Go).
- Il backend deve poter servire client differenti, inclusi dispositivi mobile o applicazioni desktop.

RNF4 Sicurezza

- Tutte le API che comportano la modifica dei dati devono essere protette tramite token di autenticazione o gestione della sessione.
- Le password devono essere memorizzate nel database esclusivamente tramite algoritmi di hashing sicuri.
- **Nota sulla produzione:** Sebbene il prototipo operi in locale su HTTP, per la messa in produzione è previsto l'uso del protocollo HTTPS per garantire la cifratura dei dati e la sicurezza delle sessioni

RNF5 Usabilità

- L'interfaccia utente deve seguire un design semplice, pulito e minimalista.
- Il sistema deve essere *responsive*, garantendo il supporto ai dispositivi mobili.

RNF6 Prestazioni

- Il tempo di risposta del sistema per le tutte operazioni in condizioni di carico standard deve essere inferiore ai 5 secondi.

RNF7 Documentazione

- Il repository GitHub del progetto deve includere: requisiti, diagrammi UML, guida all'installazione e manuale utente sintetico.

RNF8 Versionamento

- Corretto utilizzo di Git e GitHub: gestione dei branch, commit regolari e descrittivi.

Capitolo 3

Glossario

Tabella 3.1: Glossario dei termini principali del sistema Wiki Collaborativo

Termine	Definizione
Amministratore	Utente con privilegi elevati incaricato della gestione strutturale di corsi e argomenti. <i>Sinonimi: Admin.</i>
Appunto	L'unità informativa principale del sistema, contenente testo in formato Markdown o plain text renderizzato in HTML.
Argomento	Sotto-categoria tematica di un corso a cui vengono associati uno o più appunti.
Corso	Categoria tematica a cui vengono associati uno o più argomenti.
CRUD	Acronimo di Create, Read, Update, Delete; rappresenta le quattro operazioni di base gestite dalle API REST del sistema.
Markdown	Linguaggio di markup leggero utilizzato dagli studenti per la formattazione del testo degli appunti.
MVP	Pattern architetturale (Model-View-Presenter) che separa la logica di business dalla visualizzazione, come richiesto dai requisiti di progetto.
REST API	Interfaccia programmabile stateless che permette la comunicazione tramite protocollo HTTP tra il frontend (JS) e il backend (PHP).
Studente	Utente registrato che ha effettuato l'accesso e possiede i permessi per creare o modificare appunti e visualizzare la cronologia. <i>Sinonimi: Autore, Stud., StudAuth.</i>
Versione	Stato storico di un appunto salvato in seguito a una modifica o a un ripristino. Consente di navigare nella cronologia dei contenuti e di recuperare checkpoint precedenti. <i>Sinonimi: Storico, Cronologia, Checkpoint.</i>
Visitatore	Utente non registrato che non ha effettuato l'accesso. Può solo visualizzare e cercare gli appunti senza poterli creare o modificare. <i>Sinonimi: Visit.</i>

Capitolo 4

Diagrammi dei casi d'uso

Il rettangolo nel diagramma rappresenta sempre il sistema **Wiki Collaborativo per Appunti**, ovvero il **soggetto**. La modellazione dei casi d'uso è stata articolata in base ai ruoli interpretati dagli attori esterni, al fine di evidenziare il **disaccoppiamento** tra le funzionalità pubbliche e quelle riservate:

- **Visitatore**: rappresenta l'utente non autenticato che consulta il materiale.
- **Studente**: utente autenticato con permessi di contribuzione.
- **Amministratore**: ruolo preposto alla gestione strutturale di corsi e argomenti.

Sotto il profilo della modellazione, sia il ruolo **Studente** che il ruolo **Amministratore** sono rappresentati come generalizzazioni del **Visitatore**. Questa astrazione permette di indicare che entrambi i ruoli autenticati ereditano implicitamente tutti i casi d'uso di consultazione e ricerca (**RF3**, **RF4**, **RF9**) senza appesantire graficamente il diagramma con linee ridondanti. Tuttavia, si evidenziano due importanti eccezioni alla gerarchia dei permessi:

- **Esclusione del caso d'uso Registrazione (RF1)**: Una volta autenticati, né lo **Studente** né l'**Amministratore** possono accedere alla funzione di registrazione, in quanto lo stato di utente loggato è logicamente incompatibile con la creazione di un nuovo account.
- **Vincoli sulla creazione degli account**: In linea con i requisiti di sicurezza, la funzione di registrazione pubblica è limitata esclusivamente al ruolo **Studente**. Il ruolo **Amministratore** non può essere ottenuto tramite registrazione autonoma (per prevenire scalate di privilegi non

autorizzate), ma viene gestito esclusivamente a livello di persistenza tramite l'inserimento manuale o script di inizializzazione nel database.

Ho diviso i diagrammi per attore per rendere più chiara la separazione delle responsabilità e dei permessi di accesso, facilitando la tracciabilità tra i requisiti funzionali e le interazioni modellate .

4.1 Dettagli del Caso d'Uso UC1

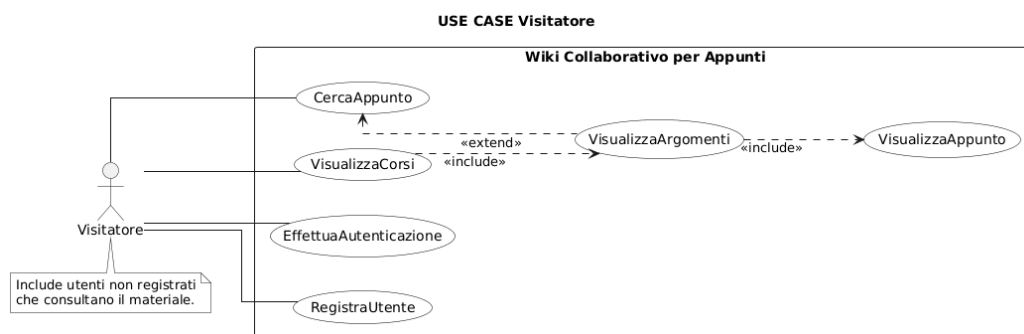


Figura 4.1: UC1 - Diagramma dei casi d'uso per l'attore Visitatore

Tabella 4.1: Specifica del caso d'uso per RF1: RegistraUtente

ID	1
Nome	RegistraUtente
Descrizione	Permette a un nuovo utente di creare un account fornendo email e password.
Attori	Visitatore.
Precondizioni	L'utente non è registrato.
Flusso Principale	1. L'utente seleziona la funzione di registrazione. 2. L'utente inserisce email e password. 2. Il sistema valida il formato dei dati (es. email valida). 4. Il sistema esegue l'hashing della password per sicurezza. 5. Il sistema memorizza i dati nel database.
Postcondizioni	Un nuovo account è stato creato con successo.
Flussi Alternativi	A1. Email già presente: Il sistema informa l'utente che l'email è già registrata e interrompe la procedura.

Tabella 4.2: Specifica del caso d'uso per RF2: EffettuaAutenticazione

ID	2
Nome	EffettuaAutenticazione
Descrizione	Consente all'utente di accedere al sistema tramite le proprie credenziali.
Attori	Visitatore.
Precondizioni	L'utente deve possedere un account registrato.
Flusso Principale	1. L'utente accede alla funzione di login. 2. L'utente inserisce email e password. 3. Il sistema confronta le credenziali con quelle nel database. 4. Il sistema avvia una sessione autenticata.
Postcondizioni	L'utente è autenticato e in base al suo ruolo può accedere alle funzioni di creazione/modifica.
Flussi Alternativi	A1. Credenziali errate: Il sistema nega l'accesso e mostra un messaggio di errore.

Tabella 4.3: Specifica del caso d'uso per RF3: VisualizzaCorsi

ID	3
Nome	VisualizzaCorsi
Descrizione	Permette di consultare l'elenco completo dei corsi disponibili nel Wiki.
Attori	Visitatore.
Precondizioni	Il sistema è accessibile.
Flusso Principale	1. L'utente accede alla pagina principale. 2. In automatico il sistema recupera i nomi dei corsi dal database. 3. Il sistema mostra l'elenco dei corsi all'utente.
Postcondizioni	L'utente visualizza l'elenco dei corsi.

Tabella 4.4: Specifica del caso d'uso per RF4: VisualizzaArgomenti

ID	4
Nome	VisualizzaArgomenti
Descrizione	Permette di navigare tra gli argomenti associati a un corso specifico.
Attori	Visitatore.
Precondizioni	L'utente ha selezionato un corso (3).
Flusso Principale	1. L'utente seleziona un corso. 2. Il sistema recupera gli argomenti legati a quel corso. 3. Il sistema mostra la lista degli argomenti di quel corso.
Postcondizioni	L'utente visualizza gli appunti collegati a quell'argomento.

Tabella 4.5: Specifica del caso d'uso per RF5: VisualizzaAppunto

ID	5
Nome	VisualizzaAppunto
Descrizione	Mostra il contenuto di un appunto in formato HTML renderizzato.
Attori	Visitatore.
Precondizioni	L'utente ha selezionato un corso e un argomento (4).
Flusso Principale	1. L'utente seleziona un appunto. 2. Il sistema recupera il testo (Markdown/Plain text) dal database. 3. Il sistema converte il testo in HTML e lo visualizza.
Postcondizioni	L'utente visualizza l'appunto renderizzato.

Tabella 4.6: Specifica del caso d'uso per RF9: CercaAppunto

ID	9
Nome	CercaAppunto
Descrizione	Permette la ricerca di appunti tramite titolo, contenuto o corso.
Attori	Visitatore.
Precondizioni	Il sistema è operativo.
Flusso Principale	1. L'utente inserisce una stringa nella funzione di ricerca. 2. Il sistema interroga il database cercando corrispondenze nei titoli. 3. Il sistema mostra i risultati filtrati.
Postcondizioni	L'utente seleziona un appunto dai risultati (prosegue con 5).

4.2 Dettagli del Caso d'Uso UC2

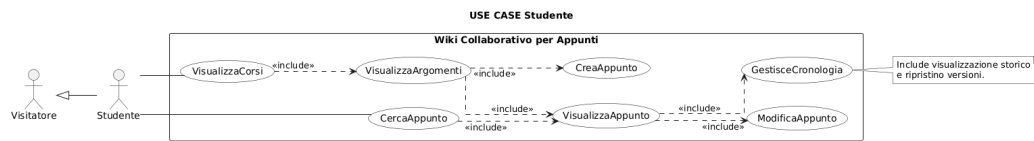


Figura 4.2: UC2 - Diagramma dei casi d'uso per l'attore Studente

Tabella 4.7: Specifica del caso d'uso per RF6: CreaAppunto

ID	6
Nome	CreaAppunto
Descrizione	Permette a un utente autenticato di inserire un nuovo appunto nel sistema associandolo a un corso e a un argomento specifico.
Attori	Studente.
Precondizioni	L'utente ha effettuato il login (2) e si trova nella sezione di visualizzazione degli appunti per argomento (4).
Flusso Principale	1. L'utente seleziona la funzione "Crea Nuovo Appunto". 2. L'utente inserisce il titolo dell'appunto. 3. L'utente inserisce il contenuto testuale (Markdown o Plain Text). 4. L'utente conferma il salvataggio. 5. Il sistema valida i dati, memorizza l'appunto e associa l'ID dell'autore.
Postcondizioni	Il nuovo appunto è persistente nel database.
Flussi Alternativi	A1. Dati mancanti: Se il titolo è vuoto, il sistema non procede al salvataggio.

Tabella 4.8: Specifica del caso d'uso per RF7: ModificaAppunto

ID	7
Nome	ModificaAppunto
Descrizione	Consente a un utente autenticato di modificare il testo di un appunto esistente. Ogni modifica genera automaticamente una nuova versione nella cronologia.
Attori	Studente.
Precondizioni	L'utente è autenticato e ha visualizzato l'appunto che intende modificare (5).
Flusso Principale	1. L'utente esegue apporta al testo le modifiche desiderate. 2. L'utente seleziona la funzione "Salva Nuova Versione" . 3. Il sistema salva le modifiche e genera automaticamente una nuova voce nella cronologia con timestamp e ID autore.
Postcondizioni	La versione corrente dell'appunto è aggiornata; la versione precedente è conservata nello storico (RF8).
Flussi Alternativi	A1. Annullamento: L'utente esce dall'editor senza salvare; il sistema non apporta modifiche e non crea nuove versioni.

Tabella 4.9: Specifica del caso d'uso per RF8: GestisceCronologia

ID	8
Nome	GestisceCronologia
Descrizione	Permette di visualizzare l'elenco delle versioni precedenti di un appunto e di effettuarne il ripristino.
Attori	Studente.
Precondizioni	L'appunto selezionato ha subito almeno una modifica (esiste uno storico versioni), l'utente si trova nella visualizzazione dell'appunto (5).
Flusso Principale	1. L'utente seleziona la voce "Vedi Cronologia". 2. Il sistema mostra l'elenco delle versioni (timestamp e autore della modifica). 3. L'utente seleziona una versione precedente per visualizzarla. 4. L'utente richiede il ripristino della versione selezionata. 5. Il sistema sovrascrive la versione attuale con quella scelta, rendendola la più recente.
Postcondizioni	L'appunto viene riportato allo stato della versione selezionata.
Flussi Alternativi	A1. Sola consultazione: L'utente visualizza una versione passata ma non effettua il ripristino.

4.3 Dettagli del Caso d'Uso UC3

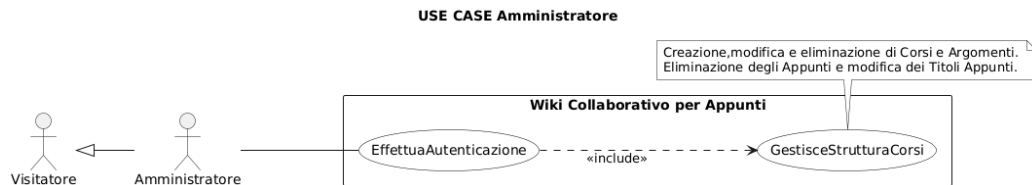


Figura 4.3: UC3 - Diagramma dei casi d'uso per l'attore Amministratore

Tabella 4.10: Specifica del caso d'uso per RF2: EffettuaAutenticazione (Admin)

ID	2
Nome	EffettuaAutenticazione (Admin)
Descrizione	Consente all'Amministratore di accedere all'area gestionale del Wiki tramite credenziali riservate.
Attori	Amministratore.
Precondizioni	Il sistema è operativo e l'Amministratore possiede credenziali valide memorizzate tramite hashing.
Flusso Principale	1. L'Amministratore inserisce email e password nella pagina di login. 2. Il sistema invia i dati al backend tramite richiesta POST. 3. Il backend valida le credenziali confrontandole con i dati nel DB. 4. Il sistema avvia una sessione con privilegi elevati (Admin).
Postcondizioni	L'Amministratore è autenticato e viene reindirizzato alla dashboard di gestione (11).
Flussi Alternativi	A1. Credenziali non valide: Il sistema nega l'accesso e restituisce un errore JSON (status 401 Unauthorized).

Tabella 4.11: Specifica del caso d'uso per RF11: GestisceStrutturaCorsi

ID	11
Nome	GestisceStrutturaCorsi
Descrizione	Permette all'Amministratore di creare, modificare o eliminare corsi e i relativi argomenti associati.
Attori	Amministratore.
Precondizioni	L'Amministratore ha effettuato con successo l'autenticazione (2).
Flusso Principale	1. L'Amministratore seleziona "Gestione Corsi" dalla dashboard. 2. Il sistema mostra l'elenco dei corsi esistenti recuperati dal DB. 3. L'Amministratore sceglie di inserire un nuovo corso o di modificarne uno esistente. 4. L'Amministratore inserisce, modifica e elimina corsi e argomenti. L'amministratore modifica i titoli degli appunti e elimina gli appunti. 5. Il sistema invia i dati al backend tramite REST API. 6. Il backend aggiorna le tabelle relazionali nel database.
Postcondizioni	La nuova struttura dei corsi è salvata in modo persistente e visibile a tutti gli utenti (RF3 , RF4 , RF5).

4.4 Matrice di tracciabilità

Tabella 4.12: Matrice di tracciabilità

Requisito Funzionale	UC1 (Visit.)	UC2 (Stud.)	UC3 (Admin)
RF1 (Registrazione)	X		
RF2 (Login/Logout)		X	X
RF3 (Lista corsi)	X		
RF4 (Lista Argomenti)	X		
RF5 (Visualizza appunto)	X		
RF6 (Crea appunto)		X	
RF7 (Modifica appunto)		X	
RF8 (Gestione Cronologia)		X	
RF9 (Ricerca)	X		
RF10 (Gestione Admin.)			X

Capitolo 5

Progettazione

La fase di progettazione traduce i requisiti analizzati in una struttura tecnica definita, seguendo un approccio top-down. Inizialmente viene definita l'architettura macroscopica del sistema, per poi scendere nel dettaglio dei singoli componenti e delle loro interazioni dinamiche.

5.1 Architettura Logica (Sottosistemi)

In conformità con lo sforzo di progettazione ad alto livello richiesto, il sistema è stato suddiviso in **sottosistemi a compartimenti stagni**. Questa scelta garantisce l'indipendenza funzionale e facilita la manutenzione, permettendo a ogni modulo di evolvere senza impattare direttamente sugli altri.

L'architettura logica si articola in tre macro-moduli principali:

- **Sottosistema di Presentazione (Frontend)**: Implementa il pattern **Model-View-Presenter (MVP)** in modalità *Passive View*, assicurando che la logica di visualizzazione sia totalmente disaccoppiata dallo stato dei dati. La *View* notifica gli eventi al *Presenter*, il quale coordina l'aggiornamento del *Model* locale.
- **Sottosistema di Logica di Business (Backend)**: Agisce come coordinatore centrale. Include i moduli *Core* per l'instradamento delle richieste tramite *Router*, e i *Table Data Gateway* per l'incapsulamento della logica *SQL*. La sicurezza autoritativa è delegata a *Protection Proxies*, mentre la generazione dinamica delle query è affidata al pattern *Strategy*.
- **Sottosistema di Persistenza (Database)**: Modella la base di dati relazionale `wiki_db`. L'interazione è mediata dall'astrazione *PDO*.

L'accesso a questo modulo è mediato interamente dal backend tramite il pattern *Table Data Gateway*.

La Figura 5.1 illustra questa gerarchia attraverso un **Package Diagram** UML. Le frecce di dipendenza mostrano come il Frontend dipenda dal Backend per il recupero dei dati, e quest'ultimo dipenda dal Database per la persistenza, rispettando il principio del disaccoppiamento.

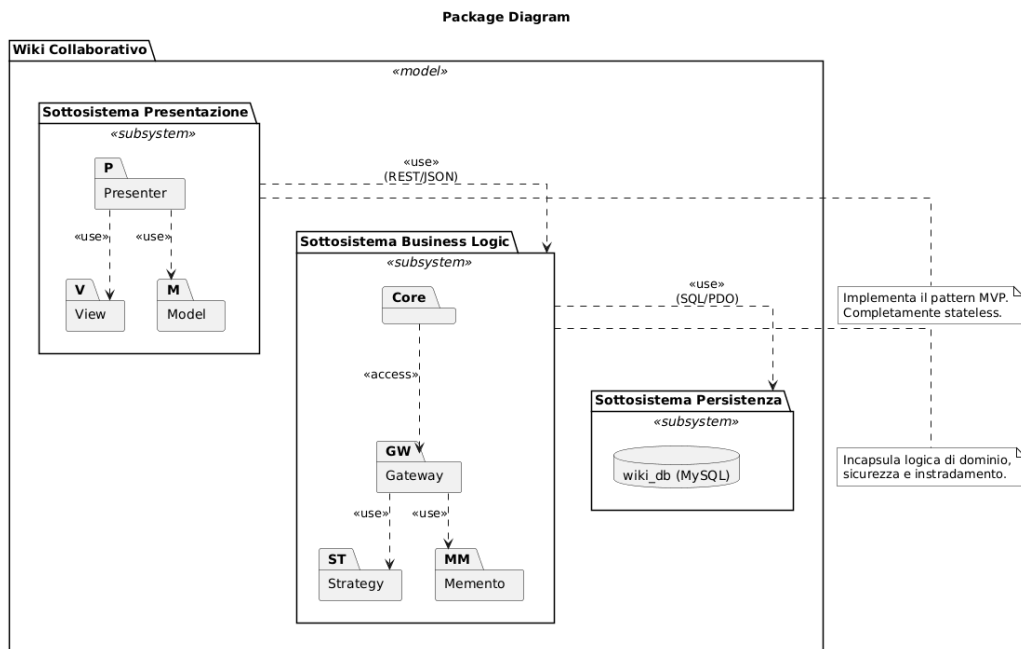


Figura 5.1: Diagramma dei pacchetti: Architettura logica a sottosistemi.

5.2 Architettura Fisica (Deployment)

L'architettura fisica del sistema è basata sulla tecnologia di containerizzazione **Docker**, che permette di isolare i sottosistemi in ambienti di esecuzione (nodi) indipendenti e riproducibili. La configurazione è definita tramite il file `docker-compose.yml`, che orchestra tre container distinti:

1. **Container Frontend:** Utilizza l'immagine `httpd:alpine`. Espone il servizio sulla porta **8080** del PC-Host, servendo i file statici (HTML/CSS) e gli script JavaScript al browser dell'utente.
2. **Container Backend:** Basato sull'immagine `php:8.2-apache`. Risponde sulla porta **8000** e gestisce la logica PHP. Include le estensioni `pdo_mysql` necessarie per il dialogo con il database.

3. **Container Database:** Utilizza `mysql:8.0` sulla porta interna **3306**. La persistenza è garantita da un volume dedicato (`db_data`), assicurando che i dati sopravvivano al ciclo di vita del container.

Tale configurazione soddisfa il requisito **RNF3 (Indipendenza)**, rendendo il backend e il frontend effettivamente intercambiabili con tecnologie alternative senza compromettere il funzionamento complessivo.

5.3 Processo di Sviluppo e Gestione

5.3.1 Pianificazione delle Attività (Diagramma di GANTT)

La pianificazione non segue una rigida successione di fasi, ma è articolata in task basati su obiettivi di progettazione. Ogni task comprende una sotto-fase di modellazione seguita dal refactoring del codice esistente per l'integrazione dei nuovi pattern.

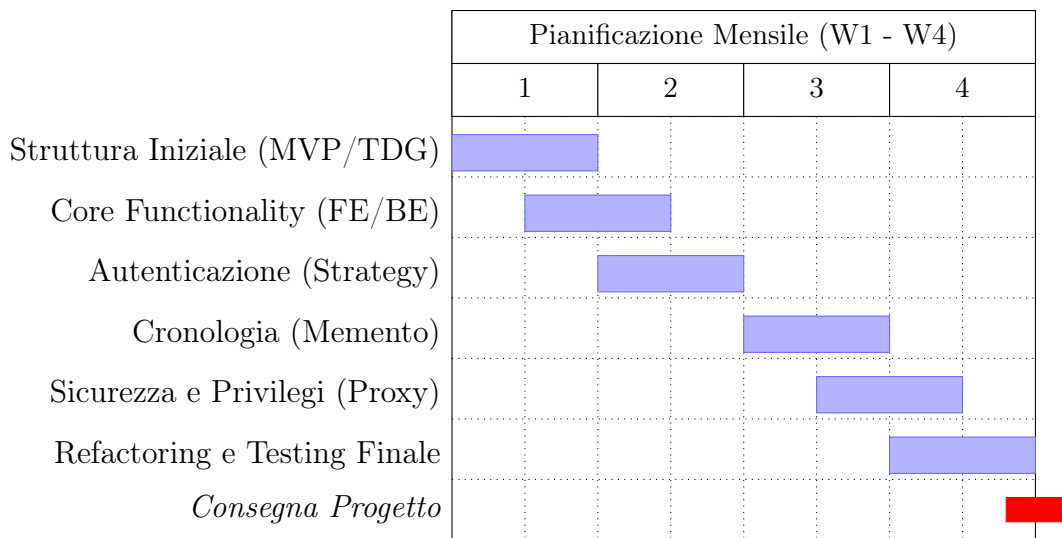


Diagramma di Gantt

5.3.2 Metodologia di Sviluppo

Lo sviluppo è stato condotto seguendo una metodologia **Agile/Scrum** adattata al singolo sviluppatore, caratterizzata da iterazioni brevi e rilasci incrementali di funzionalità (*MVP*). A differenza dei modelli tradizionali, la progettazione architettonica non è stata completata interamente in modo preliminare, ma è evoluta attraverso cicli di **Refactor Mercilessly**. Questo ha permesso di scoprire la necessità di pattern specifici (come *Memento*

o *Proxy*) solo a fronte di reali requisiti emersi durante la codifica, garantendo un design pulito e privo di sovra-progettazione (*YAGNI*). La coerenza del codice è stata mantenuta tramite l'uso di **Git** per il controllo di versione e **Docker** per garantire l'uniformità dell'ambiente di esecuzione.

5.3.3 Approccio allo Sviluppo

Il sistema è stato suddiviso in moduli logici ("tronconi"), i quali rispecchiano la strategia di **branching** adottata nel repository Git per isolare lo sviluppo delle singole funzionalità prima del merge nel ramo principale. Seguendo un approccio di **sviluppo incrementale**, ogni passaggio è stato preceduto da una fase di design specifica per garantire che l'implementazione non degradasse la qualità architetturale complessiva.

- **Struttura Iniziale:** Definizione dei pilastri (**MVP** e **Table Data Gateway**) per la separazione *FE/BE* e l'astrazione del DB.
- **Consultazione Appunti:** Stabilite le basi REST API e la gestione del File System.
- **Autenticazione:** Introduzione del pattern **Strategy** per gestire i criteri di filtraggio utenti e l'hash delle password.
- **Cronologia:** Implementazione del pattern **Memento** per il salvataggio atomico degli stati storici sul file system.
- **Privilegi e Sicurezza:** Integrazione del pattern **Protection Proxy** per centralizzare l'autorizzazione senza inquinare la logica dei Gateway.
- **Refactoring Finale:** Scomposizione del **God Object** `index.php` in una gerarchia di **Controller** e **Router**, ottimizzando la manutenibilità e la "salute" del codice sorgente, abbellimenti al FE, varie fix e aggiunta di qualche funzione extra(come l'anteprima live e il download in pdf).

5.4 Diagrammi delle Classi

I diagrammi delle classi descrivono i tipi di oggetti del sistema e le loro **relazioni statiche**. Essi formalizzano la struttura del software dettagliando attributi, operazioni e vincoli di ogni componente, comprese le classi che ne definiscono il **comportamento**.

5.4.1 Diagramma delle Classi FE

Il diagramma illustra l'architettura del client basata sul pattern MVP (Model-View-Presenter), finalizzata alla separazione tra logica di presentazione e visualizzazione.

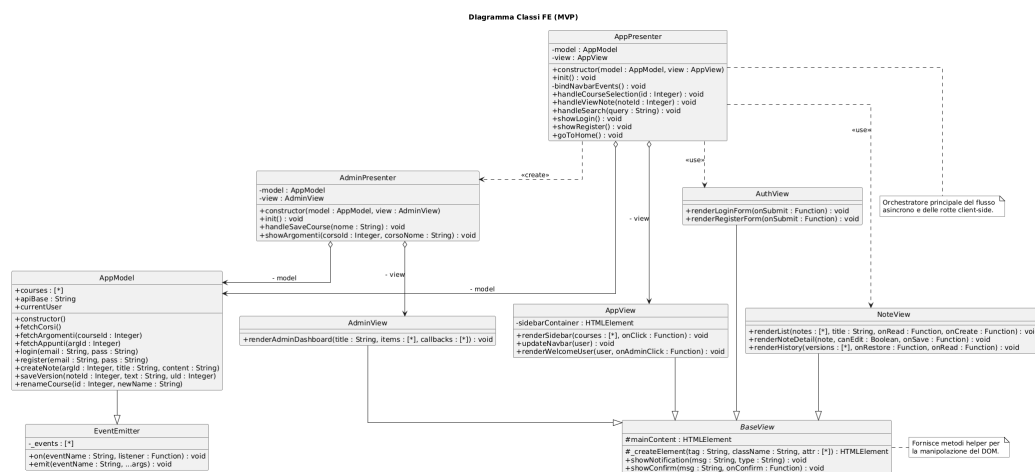


Figura 5.2: FE

5.4.2 Diagramma delle classi BE Sottosistema: Bootstrap & Orchestration

Il diagramma delle classi relativo alla fase di *Bootstrap* formalizza l'architettura logica del *Composition Root*, in cui la classe **Bootstrap** coordina l'intero setup del sistema attraverso relazioni di tipo «create» verso il **Router** e la **DatabaseFactory**. In questo contesto, il meccanismo di instradamento delle richieste è modellato come una *Usage Dependency* («use») tra il **Router** e la gerarchia dei controller, riflettendo fedelmente la natura transitoria dell'istanziatura dinamica che avviene all'interno del metodo `dispatch()`. Al fine di garantire il riuso del codice e la corretta gestione della *Dependency Injection*, è stata introdotta la classe astratta **BaseController** con funzione di *Layer Supertype*, necessaria a fattorizzare il riferimento al database e ai *Gateway* comuni a tutte le specializzazioni concrete.

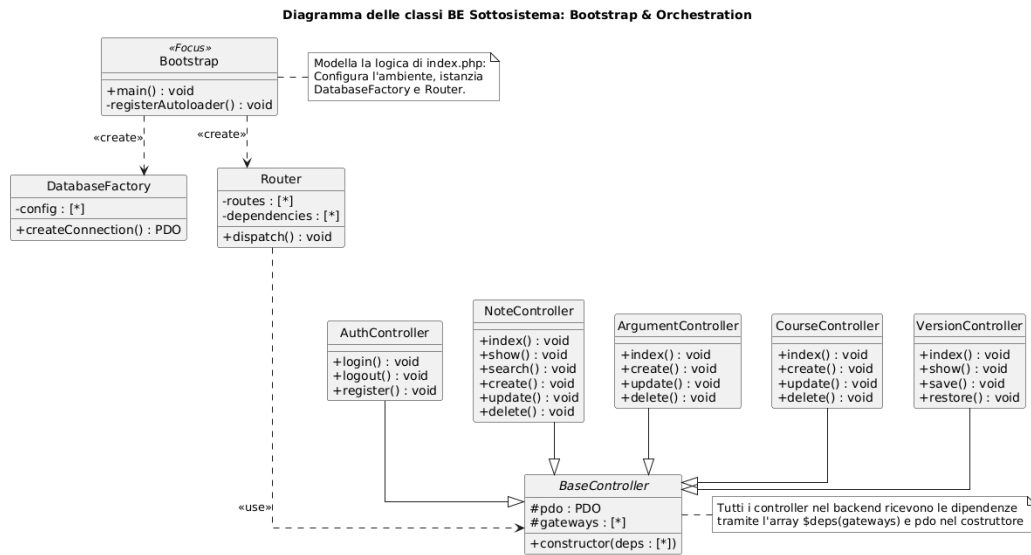


Figura 5.3: Diagramma delle classi BE Sottosistema: Bootstrap & Orchestration

5.4.3 Diagramma delle classi BE Sottosistema: Persistence & Security

Il diagramma del sottosistema di persistenza e sicurezza formalizza l'architettura di accesso allo strato dei dati, strutturata mediante l'integrazione dei pattern *Table Data Gateway* e *Protection Proxy*. Il cuore strutturale è rappresentato dalla gerarchia che discende dalla classe astratta **AbstractGateway**, la quale centralizza il riferimento protetto all'oggetto PDO e fornisce i metodi necessari per la composizione sicura delle query SQL tramite *Query Object*. In conformità ai requisiti di sicurezza (**RNF4**), l'accesso operativo ai dati è mediato da una serie di classi **Proxy** (es. **NotesGatewayProxy**) che realizzano le interfacce del sottosistema. Tali componenti permettono di intercettare ogni chiamata e verificare i privilegi dell'utente in sessione prima di delegare l'esecuzione effettiva, garantendo così che operazioni sensibili come la creazione o l'eliminazione siano riservate ai soli ruoli autorizzati. Il modulo **CascadeService** agisce come coordinatore per le procedure di eliminazione complessa, orchestrando l'interazione tra gateway differenti.

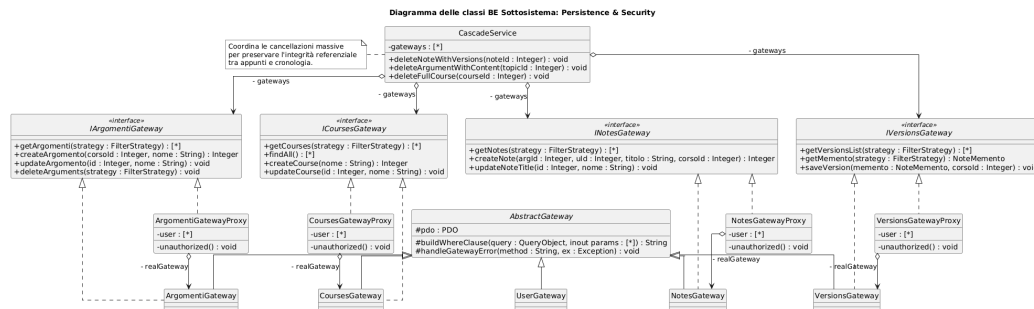


Figura 5.4: Diagramma delle classi BE Sottosistema: Persistence & Security

5.4.4 Diagramma delle classi BE Sottosistema: Strategy & Query Object

Il diagramma delle classi relativo al filtraggio delle risorse formalizza l'architettura utilizzata per la generazione dinamica dei criteri di ricerca, basata sull'integrazione dei pattern *Strategy* e *Query Object*. L'interfaccia *FilterStrategy* funge da astrazione centrale, definendo il contratto operativo *buildCriteria()* che permette di estendere il sistema con nuovi algoritmi di filtraggio (es. *SearchFilter* o *CourseFilter*) senza modificare i client esistenti, rispettando il principio *Open/Closed*. Il disaccoppiamento tra la logica di dominio e la sintassi SQL è garantito dalla classe *QueryObject*. Tale struttura permette alle strategie concrete di popolare l'oggetto di query con requisiti logici (come l'operatore *LIKE* o *IN*), che verranno successivamente interpretati dall'infrastruttura di persistenza per produrre *prepared statements* sicuri, assicurando così l'integrità del sistema contro attacchi di *SQL Injection*.

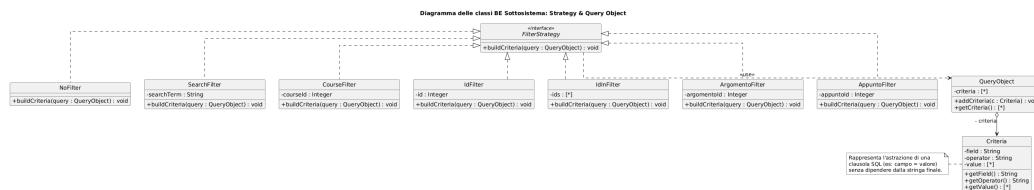


Figura 5.5: Diagramma delle classi BE Sottosistema: Strategy & Query Object

5.4.5 Diagramma delle classi BE Sottosistema: Gestione Stato e Cronologia

Il diagramma delle classi relativo alla persistenza e al ripristino degli appunti formalizza l'applicazione del pattern *Memento*. La classe **Note** assume il ruolo di *Originator*, detenendo lo stato interno dell'appunto e fornendo i metodi `saveToMemento()` e `restoreFromMemento()` per la generazione e il ripristino dei checkpoint. Il legame tra l'originatore e il componente **NoteMemento** è modellato tramite una *Usage Dependency*, a indicare che la responsabilità dell'istanziamento dello stato spetta esclusivamente alla classe **Note**. **NoteMemento** definisce i propri attributi con visibilità privata e li mette a disposizione tramite il metodo `getState()`, assicurando che lo stato sia accessibile solo per fini di persistenza o ripristino autorizzato. **VersionsGateway** funge da *Caretaker*, coordinando la custodia e il recupero dei memento dal database.

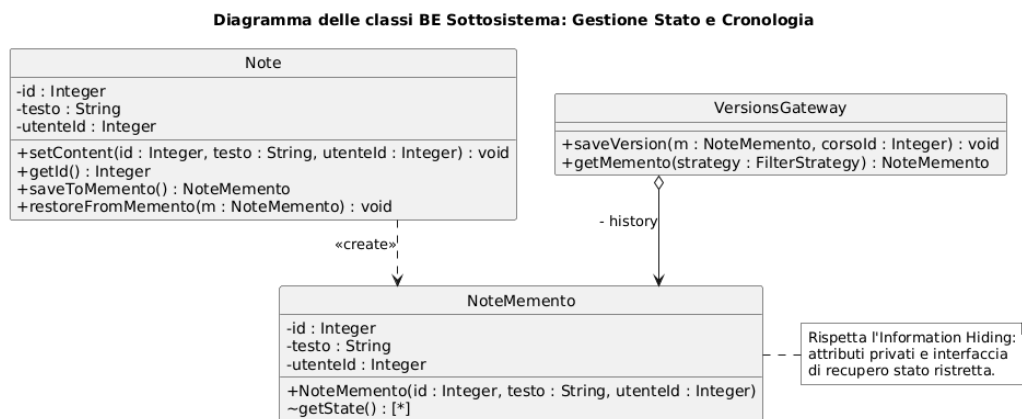


Figura 5.6: Diagramma delle classi BE Sottosistema: Gestione Stato e Cronologia

5.5 Diagrammi di Sequenza

I seguenti diagrammi mostrano l'interazione temporale tra gli elementi dell'applicativo descrivendo alcuni casi d'uso e mettendo in luce l'uso di alcuni pattern.

5.5.1 Diagramma di sequenza: RF6 (Creazione Appunto)

Il diagramma di sequenza relativo alla creazione di un nuovo appunto formalizza l'integrazione del pattern *Protection Proxy*. Il `NoteController`, operando come client dell'interfaccia `INotesGateway`, interagisce in modo trasparente con il `NotesGatewayProxy` senza conoscere la natura reale dell'oggetto di persistenza. Il cuore del meccanismo di sicurezza risiede nella logica del proxy, il quale intercetta l'invocazione del metodo `createNote()` e verifica che l'utente in sessione possieda il ruolo di "studente". Solo in caso di esito positivo del controllo autoritativo, il proxy delega l'operazione al `NotesGateway` reale di creazione dell'appunto.

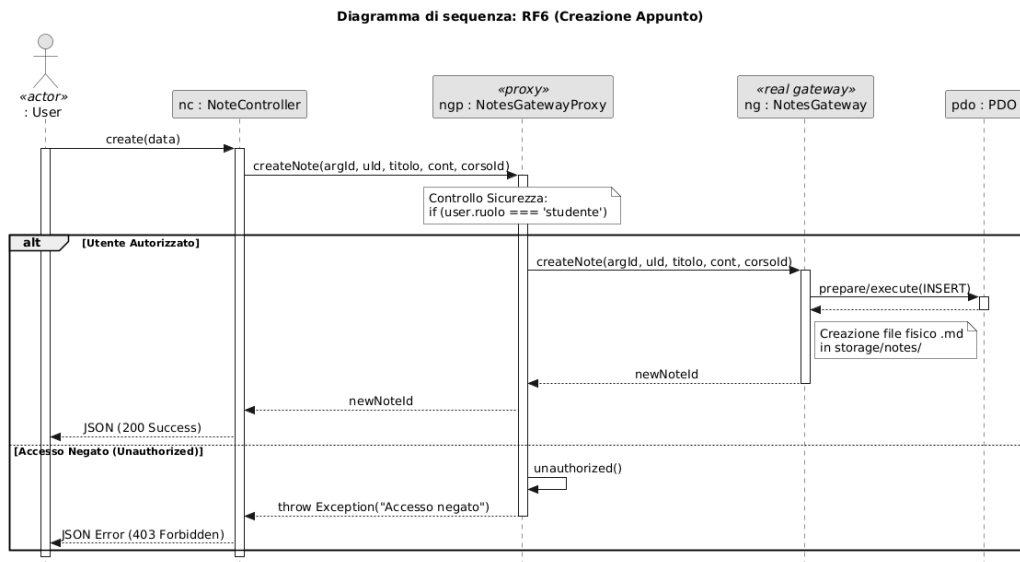


Figura 5.7: Diagramma di sequenza: RF6 (Creazione Appunto)

5.5.2 Diagramma di sequenza: RF8 (parte Salvataggio Nuova Versione)

Il diagramma di sequenza illustra la procedura di creazione di un checkpoint per un appunto applicando il pattern *Memento*. Il processo è coordinato dal `VersionController`, il quale, dopo aver aperto una transazione tramite l'astrazione `PDO`, aggiorna l'oggetto `Note` (*Originator*) e ne richiede la cattura dello stato tramite il metodo `saveToMemento()`. Quest'ultimo istanza un oggetto `NoteMemento` che incapsula le informazioni correnti in modo immutabile. Il `VersionsGateway` agisce come *Caretaker*, ricevendo il memento

e accedendo alla sua interfaccia “stretta” tramite il metodo `getState()` per persistere i dati sul database.

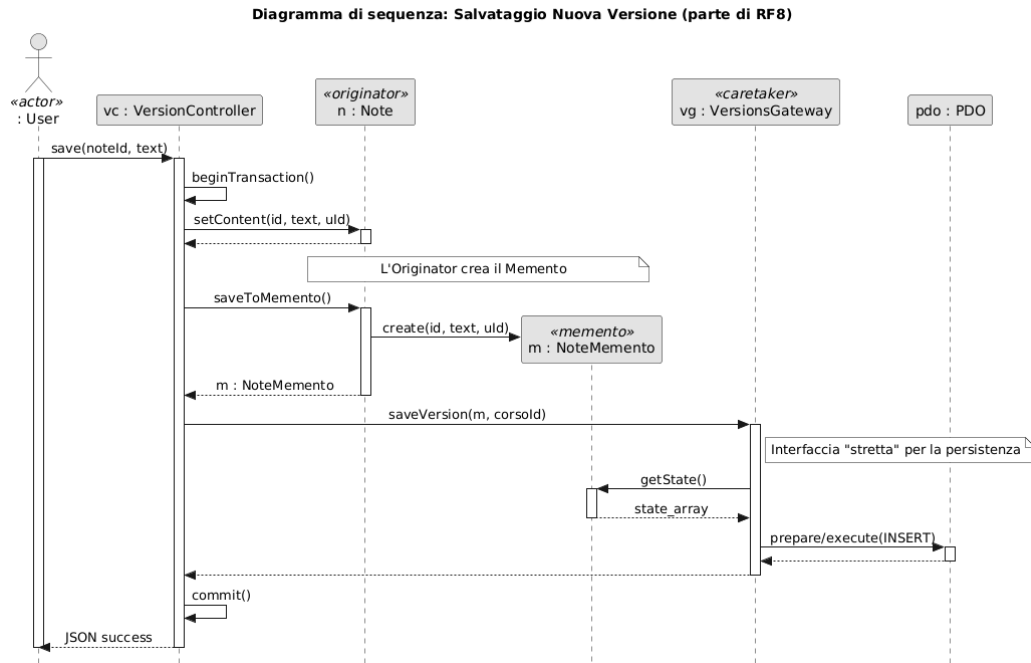


Figura 5.8: Diagramma di sequenza: RF8 (parte Salvataggio Nuova Versione)

5.5.3 Diagramma di sequenza: RF9 (Ricerca Appunti)

Il diagramma di sequenza per la ricerca (RF9) illustra l'applicazione del pattern **Strategy**, garantendo il **disaccoppiamento** tra logica di controllo e accesso ai dati. Il `NoteController` istanzia la strategia concreta `SearchFilter` e la passa al `NotesGateway` tramite il metodo `getNotes()`. Il gateway delega alla strategia il popolamento del `QueryObject` mediante `buildCriteria()`, applicando l'operatore `LIKE` sul titolo. Infine, il metodo `buildWhereClause` trasforma i criteri in un **prepared statement** SQL sicuro contro la **SQL Injection**. Questa architettura rispetta il principio **Open/Closed**, permettendo di aggiungere nuovi filtri senza modificare la struttura del gateway.

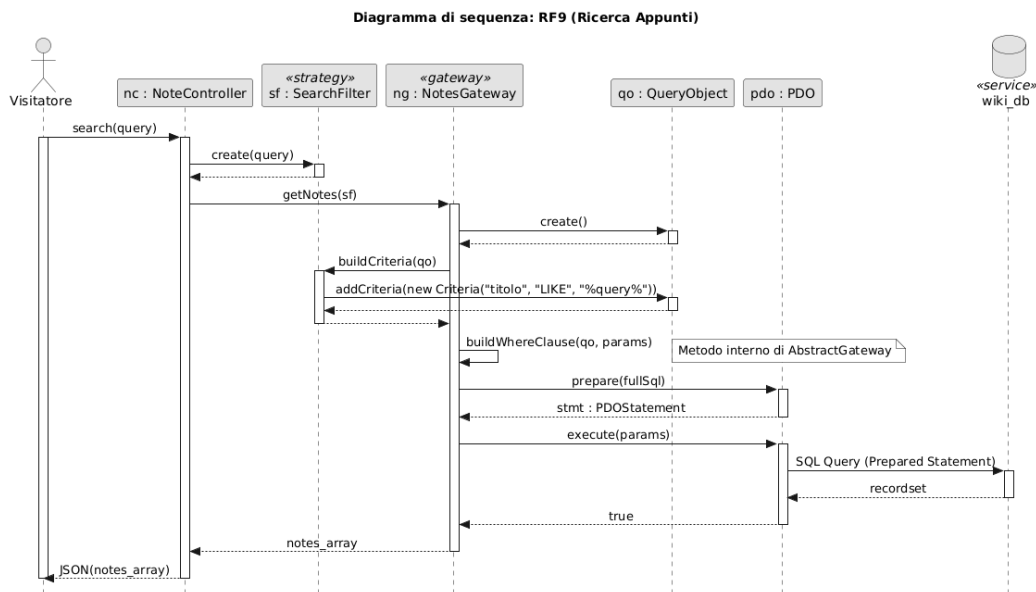


Figura 5.9: Diagramma di sequenza: RF9 (Ricerca Appunti)

5.6 Diagrammi di Attività

I diagrammi delle attività descrivono il flusso di controllo e le informazioni dei processi, supportando la modellazione di comportamenti paralleli. Nella progettazione software, questi grafici illustrano algoritmi e operazioni attraverso nodi, archi e partizioni (*swimlanes*) per suddividere le responsabilità tra i componenti.

5.6.1 Diagramma delle Attività: Login (RF2)

Il diagramma delle attività per il **login (RF2)** illustra la verifica dell'identità. Il flusso parte dal **AppPresenter** (validazione sintattica) e passa al **AppModel**. Nel backend, l'**AuthController** coordina lo **UserGateway**, che applica la **Strategy EmailFilter** per il recupero dei dati. In linea con i requisiti di **sicurezza (RNF4)**, il sistema confronta l'**hash sha256** della password fornita con quello memorizzato. Eventuali errori sono gestiti da nodi di decisione che restituiscono uno stato **HTTP 401 (Unauthorized)**, mentre il successo determina l'avvio della sessione e il reindirizzamento dell'utente.

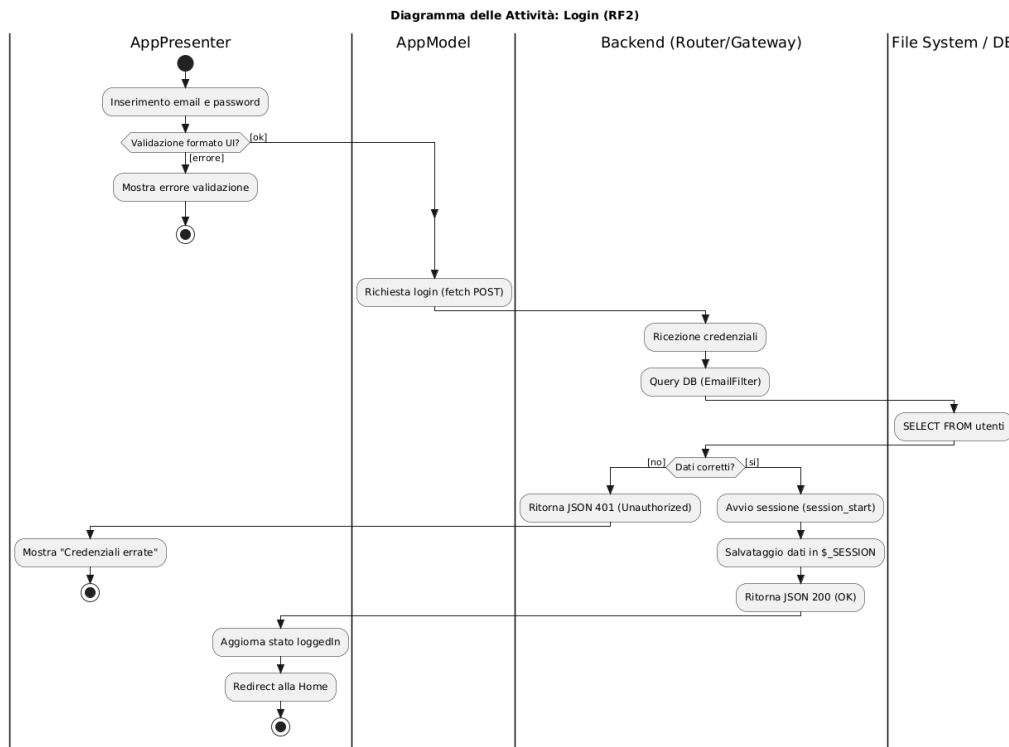


Figura 5.10: Diagramma delle Attività: Login (RF2)

5.6.2 Diagramma delle Attività: Visualizza Appunto (RF5)

Il seguente diagramma delle attività illustra il flusso di recupero di un appunto, caratterizzato dalla scomposizione della risorsa tra database relazionale e *file system*. Il processo è innescato dal **AppPresenter** e mediato dal **AppModel** tramite una richiesta asincrona allo strato di *backend*. Il **NotesGateway** esegue una query SQL per estrarre i metadati e il percorso relativo del file; successivamente, una struttura di controllo verifica l'esistenza fisica del documento Markdown nel *file system*. Il punto critico del processo risiede nell'"Integrazione Dati", fase in cui il gateway aggrega il record del database con il contenuto testuale letto dal disco prima di veicolare la risposta JSON completa verso il *frontend*, garantendo la tracciabilità del requisito RF5 e la gestione degli errori in caso di file mancante.

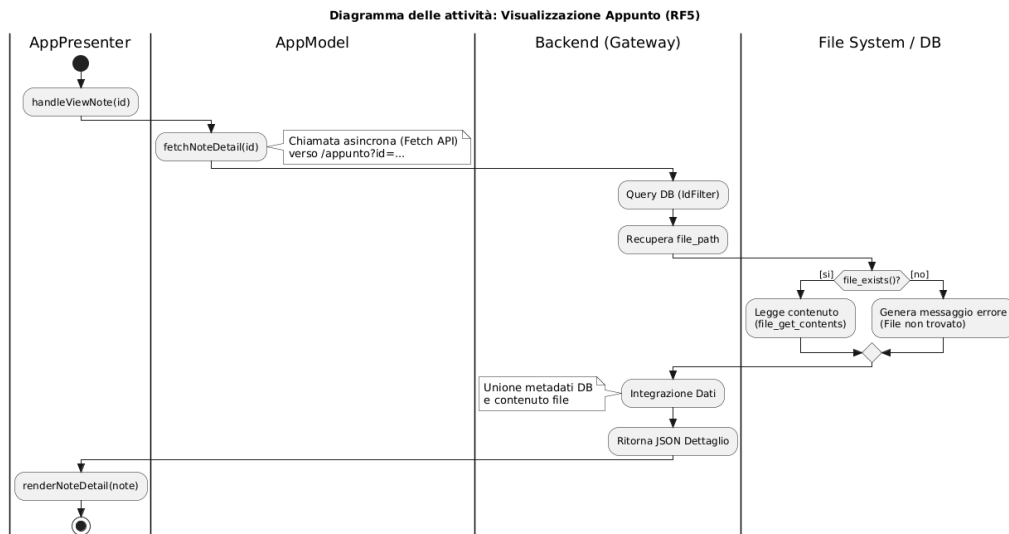


Figura 5.11: Diagramma delle Attività: Visualizza Appunto (RF5)

5.7 Progettazione dei Dati

Il sistema adotta un approccio di persistenza ibrido per ottimizzare la gestione di contenuti testuali variabili. Il database relazionale MySQL è incaricato della gestione della struttura logica. Il contenuto effettivo degli appunti e delle loro versioni non viene salvato nelle tabelle, ma archiviato direttamente nel file system del server. All'interno del database, i campi `file_path` fungono da puntatori verso i file Markdown fisici, permettendo di unire la velocità di accesso al disco per file voluminosi con la coerenza dei metadati garantita dallo strato SQL.

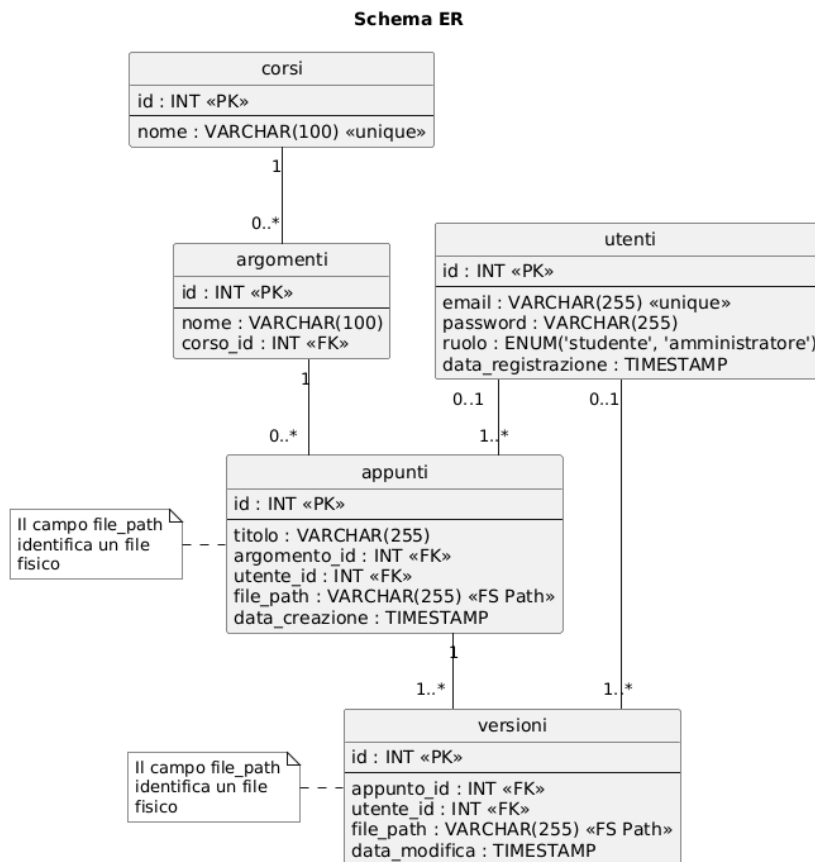


Figura 5.12: Schema Entity-Relationship

5.7.1 Organizzazione del File System

I file sono organizzati in una struttura a directory che riflette la gerarchia logica del catalogo, facilitando la manutenzione e scalabilità del sistema. La radice del repository è la cartella **notes** (sotto **backend/storage/**), strutturata come segue:

```

notes/
├── [ID_Corso]/
│   ├── [ID_Appunto].md ..... Versione attuale
│   └── versions/
│       ├── [ID_Appunto]/
│       │   ├── v1.md ..... Versioni numerate progressivamente
│       │   └── v2.md

```

L'utilizzo degli ID numerici come nomi di directory e file offre molteplici vantaggi tecnici:

1. **Disaccoppiamento:** Il File System è indipendente dai titoli dei corsi o degli appunti, che possono essere rinominati nel database senza richiedere modifiche ai file.
2. **Performance:** La segmentazione in sottodirectory previene il degrado delle prestazioni tipico dei file system con un numero eccessivo di file in un'unica cartella (flat directory).
3. **Integrità:** La separazione netta tra la cartella dei contenuti "vivi" e la directory `versions/` riduce il rischio di sovrascritture accidentali durante le operazioni di manutenzione.

5.8 Specifica REST API

La comunicazione tra il frontend (*JavaScript*) e il backend (*PHP*) del sistema Wiki Collaborativo avviene attraverso un'architettura di tipo **RESTful** (Representational State Transfer). Questa scelta garantisce un netto disaccoppiamento tra l'interfaccia utente e la logica di business, permettendo ai due componenti di evolvere indipendentemente e rendendo il sistema facilmente manutenibile e scalabile. Di seguito elenco i principi fondamentali del paradigma **REST**:

- **Risorse e URI:** Ogni entità del dominio, come *corsi*, *argomenti* e *appunti*, è trattata come una risorsa identificata univocamente da un **URI** (Uniform Resource Identifier) descrittivo e intuitivo, seguendo una struttura gerarchica coerente.
- **Protocollo Stateless:** Ogni richiesta inviata dal client al server è autocontenuta e include tutte le informazioni necessarie per l'elaborazione; al server non è richiesto di memorizzare lo stato del contesto dell'utente tra una chiamata e l'altra, favorendo la scalabilità del sistema.
- **Interfaccia Uniforme e CRUD:** Le interazioni avvengono tramite l'uso esplicito dei verbi standard **HTTP**, che mappano direttamente le operazioni **CRUD** (*Create*, *Read*, *Update*, *Delete*) sulle risorse: **GET** per il recupero (*Read*), **POST** per la creazione (*Create*), **PUT** per l'aggiornamento (*Update*) e **DELETE** per la rimozione (*Delete*).

- **Formato JSON:** Tutte le risposte prodotte dal backend sono strutturate in formato **JSON** (JavaScript Object Notation), garantendo un interscambio di dati leggero, facile da leggere per gli umani e semplice da interpretare per le macchine.

Questa interfaccia definisce il contratto formale tra client e server, basato sulla netta **Separation of Concerns**. Tale approccio assicura che il frontend possa essere teoricamente sostituito con tecnologie alternative (come applicazioni mobile o desktop) senza dover modificare la logica di persistenza e di business implementata nel backend.

5.8.1 Endpoint

Di seguito vengono dettagliati i principali endpoint.

1. Registrazione utente (RF1)

- **URL:** /register
- **Metodo:** POST
- **Parametri Query:**
 - email (obbligatorio).
 - password (obbligatorio).
- **Descrizione:** Riceve le credenziali dal frontend e crea un nuovo utente.

Esempio risposta di Successo (JSON):

```
{"status": "success", "id": 4}
```

2. Login (RF2)

- **URL:** /login
- **Metodo:** POST
- **Parametri Query:**
 - email (obbligatorio).
 - password (obbligatorio).

- **Descrizione:** Verifica la corrispondenza tra email e password hashata per avviare la sessione.

Esempio risposta di Successo (JSON):

```
{
  "status": "success",
  "user": {
    "id": 3,
    "email": "nico@studenti.unipr.it",
    "ruolo": "studente",
    "data_registrazione": "2026-01-31 18:18:59"
  }
}
```

3. Logout (RF2)

- **URL:** /logout
- **Metodo:** POST
- **Descrizione:** Invalida la sessione lato server e pulisce lo stato lato client.

4. Visualizzazione Corsi (RF3)

- **URL:** /corsi
- **Metodo:** GET
- **Descrizione:** Restituisce l'array di tutti i corsi presenti nel database.

Esempio risposta di Successo (JSON):

```
[
  { "id": 1, "nome": "Informatica" },
  { "id": 2, "nome": "Matematica" }
]
```

5. Visualizzazione argomenti di un corso (RF4)

- **URL:** /argomenti
- **Metodo:** GET
- **Parametri Query:**
 - **corso_id** (obbligatorio): Identificativo univoco del corso di riferimento. È un parametro obbligatorio affinché la richiesta vada a buon fine.
- **Descrizione:** Permette di recuperare l'elenco di tutti gli argomenti associati a un determinato corso selezionato dall'utente.

Esempio risposta di Successo (JSON):

```
[
  {
    "id": 4,
    "nome": "Meccanica",
    "corso_id": 3
  },
  {
    "id": 5,
    "nome": "Termodinamica",
    "corso_id": 3
  }
]
```

6. Recupero Appunti per Argomento (RF4,RF5)

- **URL:** /appunti
- **Metodo:** GET
- **Parametri Query:**
 - **argomento_id** (obbligatorio): Identificativo univoco dell'argomento scelto.
- **Descrizione:** Questo endpoint permette di ottenere la lista di tutti gli appunti associati a un determinato argomento. Risponde alla necessità dell'utente di navigare all'interno della gerarchia Corso → Argomento per giungere alla risorsa finale.

Esempio risposta di Successo (JSON):

```
[
  {
    "id": 4,
    "titolo": "Meccanica Classica",
    "argomento_id": 4,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\meccanica_classica.md",
    "data_creazione": "2026-01-30 14:53:36"
  },
  {
    "id": 5,
    "titolo": "Meccanica Statistica",
    "argomento_id": 4,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\meccanica_statistica.md",
    "data_creazione": "2026-01-30 14:53:36"
  }
]
```

7. Visualizzazione degli appunti (RF5)

- **URL:** /appunto
- **Metodo:** GET
- **Parametri Query:**
 - id (obbligatorio): Identificativo univoco dell'appunto scelto.
- **Descrizione:** Restituisce i metadati e il contenuto grezzo del file Markdown.

Esempio risposta di Successo (JSON):

```
{
  "id": 4,
  "titolo": "Meccanica Classica",
  "argomento_id": 4,
  "utente_id": 1,
  "file_path": "storage\\notes\\3\\meccanica_classica.md",
  "data_creazione": "2026-01-30 14:53:36",
  "contenuto": "# Meccanica classica\r\n\r\nLa **meccanica classica**[...]"
}
```

8. Ricerca (RF9)

- **URL:** /cerca
- **Metodo:** GET
- **Parametri Query:**
 - **q** (obbligatorio): Rappresenta il termine di ricerca inserito dall'utente. Il sistema backend processa la richiesta solo se la lunghezza di **q** è maggiore o uguale a 2 caratteri; in caso contrario, viene restituita una lista vuota per ottimizzare le prestazioni
- **Descrizione:** Permette di filtrare la collezione globale degli appunti cercando una corrispondenza testuale all'interno dei titoli.
- **Risposta di Successo (JSON):**

```
[
  {
    "id": 1,
    "titolo": "Introduzione al Corso",
    "argomento_id": 1,
    "utente_id": 1,
    "file_path": "storage\\notes\\1\\introduzione_al_corso.md",
    "data_creazione": "2026-01-30 14:53:36"
  },
  {
    "id": 6,
    "titolo": "Introduzione alla Termodinamica",
    "argomento_id": 5,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\introduzione_alla_termodinamica.md",
    "data_creazione": "2026-01-30 14:53:36"
  }
]
```

9. Creazione appunti (RF6)

- **URL:** /appunto/crea
- **Metodo:** POST
- **Parametri Query:**

- titolo (obbligatorio)
- argomento_id (obbligatorio)
- utente_id (obbligatorio)
- contenuto (obbligatorio)

- **Descrizione:** Permette a uno studente autenticato di creare un nuovo appunto. Il sistema salva i metadati nel database e crea fisicamente il file .md nello storage dedicato.

- **Risposta di Successo (JSON):**

```
{ "status": "success", "id": 15 }
```

10. Modifica appunti (RF7)

- **URL:** /appunto/versione/salva

- **Metodo:** POST

- **Body JSON:**

- id (obbligatorio): ID dell'appunto da modificare.
- testo (obbligatorio): Nuovo contenuto Markdown.
- utente_id (obbligatorio): ID dello studente che effettua la modifica.

- **Descrizione:** Sovrascrive il contenuto dell'appunto attuale e, contestualmente, applica il pattern *Memento* archiviando lo stato precedente come una nuova versione nella cronologia.

- **Risposta di Successo (JSON):**

```
{
  "status": "success",
  "message": "Cronologia aggiornata e modifiche salvate"
}
```

11. Gestione corsi, argomenti e appunti (RF10)

- **URL (Crea):** /corso/crea e /argomento/crea (POST)
- **URL (Modifica):** /corso/modfica, /argomento/modfica e /appunto/modfica (PUT)
- **URL (Elimina):** /corso/elimina, /argomento/elimina e /appunto/elimina (DELETE)
- **Parametri Body (Esempio Creazione):**
 - nome (obbligatorio): Nome della nuova risorsa (corso/argomento).
 - id (obbligatorio per modifiche ed eliminazioni).
- **Descrizione:** Insieme di API protette dal *Protection Proxy* riservate al ruolo amministratore. Le operazioni di eliminazione utilizzano il CascadeService per garantire la pulizia atomica di DB e File System.
- **Risposta di Successo (JSON):**

```
{ "status": "success" }
```

12. Cronologia delle versioni (RF8)

- **URL (Lista):** /appunto/storia?id=[ID_APPUNTO] (GET)
- **URL (Visualizza):** /appunto/versione/visualizza?versione_id=[ID] (GET)
- **URL (Ripristina):** /appunto/versione/ripristina (POST)
- **Parametri Body (Ripristina):**
 - versione_id (obbligatorio): ID della versione memento da promuovere.
 - utente_id (obbligatorio): ID dell'utente che richiede il ripristino.
- **Descrizione:** Gestione dello storico tramite pattern *Memento*. Il ripristino recupera lo stato dal database e dal file system, archiviando lo stato corrente prima di sovrascriverlo.
- **Risposta di Successo (JSON - Ripristina):**

```
{  
  "status": "success",  
  "message": "Versione ripristinata correttamente"  
}
```

Capitolo 6

Architettura

6.1 Design Pattern Model-View-Presenter (MVP)

Il pattern MVP separa le responsabilità in tre componenti distinti: il **Model**, che incapsula la business logic e la persistenza; la **View**, che agisce come interfaccia passiva delegando ogni decisione al Presenter; e il **Presenter**, che funge da orchestratore. A differenza del pattern MVC, la View non osserva direttamente il Model, riducendo l'accoppiamento e facilitando il testing in isolamento.

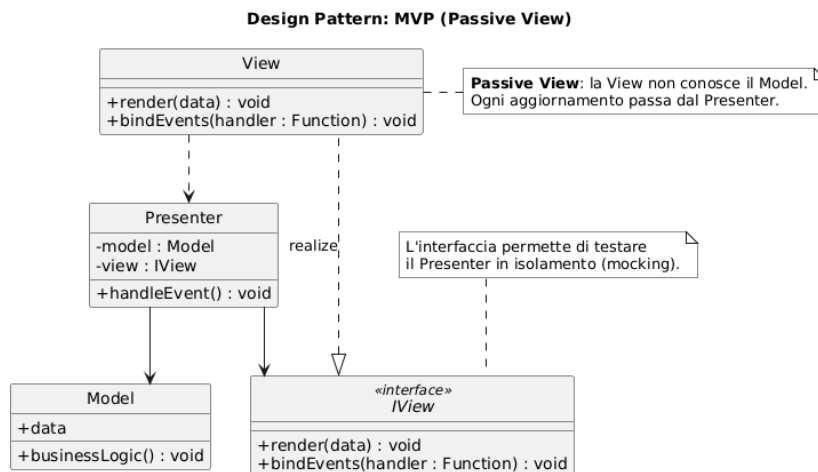


Figura 6.1: MVP

6.1.1 Mapping nel codice

- **AppPresenter e AdminPresenter (Presenter):** Fungono da mediatore; riceve gli input dalla View e coordina le richieste verso il Model, aggiornando l'interfaccia in base ai dati ricevuti.
- **AppModel (Model):** Gestisce lo stato dei dati e l'interazione con le API REST del backend tramite richieste asincrone (*Promises*).
- **BaseView, AppView, AuthView, AdminView, NoteView (View e IView):** Rappresentano lo strato di visualizzazione (*Passive View*); si occupano esclusivamente della manipolazione del DOM e del rendering dei corsi e degli appunti.

6.2 Design Pattern Table Data Gateway

Il pattern **Table Data Gateway** fornisce un oggetto che funge da gateway verso una specifica tabella del database. Esso contiene tutta la logica SQL (*SELECT, INSERT, UPDATE, DELETE*) necessaria, isolando il resto dell'applicazione dai dettagli della persistenza. In genere restituisce un **RecordSet**, ovvero una struttura dati generica che mima la natura tabulare dei dati.

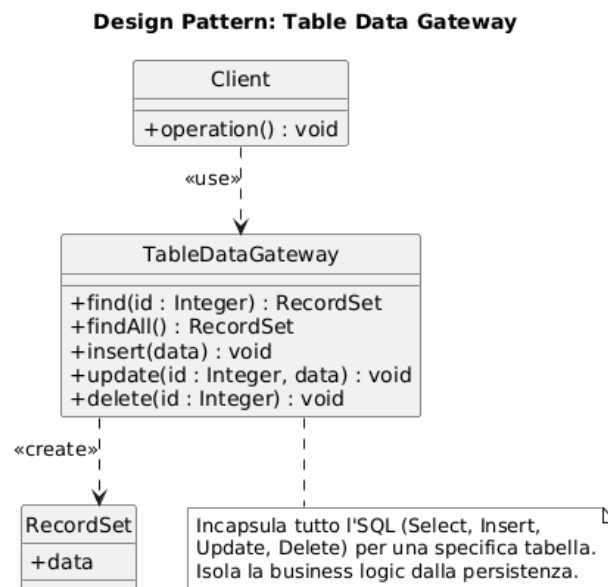


Figura 6.2: Table Data Gateway

6.2.1 Mapping nel codice

L'implementazione del sistema di persistenza segue una struttura gerarchica basata sul pattern **Table Data Gateway**, garantendo la separazione tra logica di business e accesso ai dati. Il mapping delle classi principali è articolato come segue:

- **AbstractGateway** (Classe Base): Fornisce la logica comune a tutti i *gateway*. Centralizza il metodo protetto `buildWhereClause()` il quale è fondamentale per trasformare i criteri astratti, definiti tramite il pattern *Strategy*.
- **CoursesGateway**: Classe che gestisce la tabella `corsi`.
- **ArgomentiGateway**: Classe che gestisce la tabella `argomenti`.
- **NotesGateway**: Classe che gestisce la tabella `appunti`.
- **UserGateway**: Classe che gestisce la tabella `utenti`.
- **VersionsGateway**: Classe che gestisce la tabella `versioni`. Si occupa anche della persistenza degli stati storici degli appunti, interfacciandosi con il sistema di versionamento basato sul pattern *Memento*.

6.3 Design Pattern Strategy

Il pattern **Strategy** è un pattern comportamentale utilizzato per definire una serie di algoritmi intercambiabili a runtime. Attraverso l'uso dell'aggregazione, il **Context** delega l'esecuzione del compito a un oggetto che realizza l'interfaccia **Strategy**.

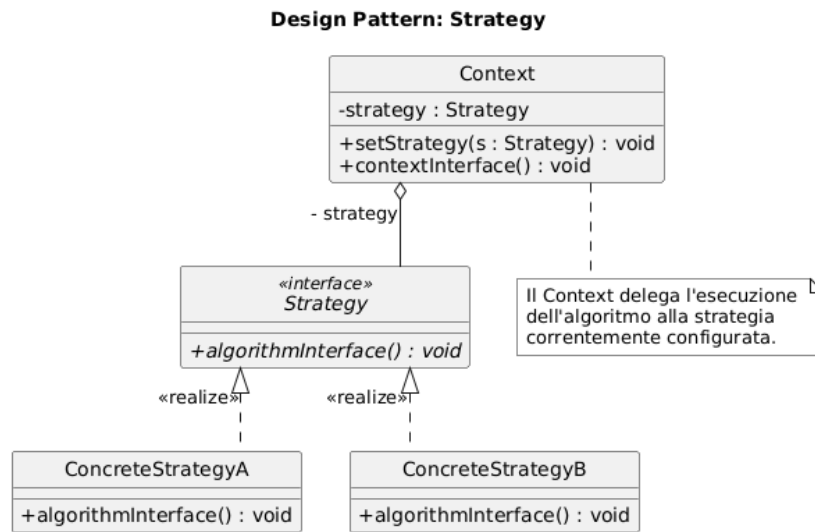


Figura 6.3: Strategy

6.3.1 Mapping nel codice

- **FilterStrategy** (Interfaccia): *Contratto* comune agli algoritmi di filtraggio. Definisce il metodo astratto `buildCriteria(QueryObject $query)`, che obbliga ogni strategia concreta a popolare un oggetto di query con i criteri specifici del dominio.
- **CourseFilter**: Implementa il filtraggio degli argomenti in base all'identificativo di un corso specifico, in ottemperanza al requisito funzionale **RF4**.
- **ArgomentoFilter**: Filtro degli appunti in base all'identificativo di un argomento specifico.
- **SearchFilter**: Implementa la logica di ricerca testuale prevista dal requisito **RF9**. Istruisce il sistema a filtrare gli appunti il cui titolo corrisponde alla stringa inserita dall'utente tramite l'operatore SQL LIKE e l'utilizzo dei caratteri jolly %.
- **IdFilter** e **EmailFilter**: Ulteriori specializzazioni del pattern utilizzate rispettivamente per recuperare il dettaglio di un singolo appunto (**RF5**) o per estrarre le informazioni di un utente durante la procedura di *login* tramite il campo `email` memorizzato nel database.
- **NoFilter**: Utilizzata per soddisfare il contratto **Strategy** quando non c'è bisogno di fare alcun filtro.

- **AppuntoFilter**: Permette di isolare le versioni appartenenti a uno specifico appunto, facilitando il recupero della cronologia (RF8).
- **IdInFilter**, **ArgomentiIdInFilter** e **AppuntoIdInFilter**: Strategie ottimizzate per la gestione di collezioni di identificativi tramite l'operatore IN. Sono componenti critici per il **CascadeService**, permettendo la rimozione atomica e sicura di record correlati in scenari di cancellazione massiva (RF10).

6.4 Design Pattern Memento

La gestione dello stato adotta il pattern **Memento**, garantendo l'incapsulamento dell'**Originator** tramite un'interfaccia "stretta" (*narrow interface*). L'originatore crea i checkpoint mediante una *Usage Dependency* («create»), mentre il **Caretaker** orchestra la cronologia dei dati. L'uso della molteplicità [*] per la proprietà - **history** assicura la gestione di una sequenza illimitata di stati, supportando operativamente l'*undo* multi-livello.

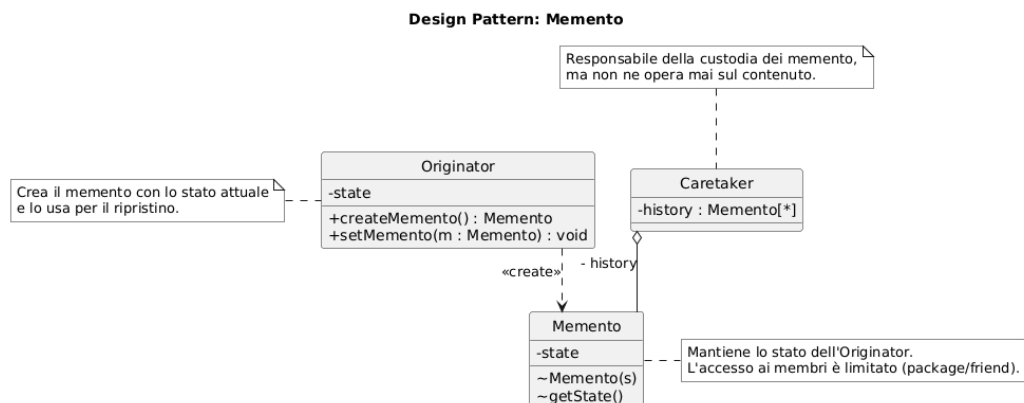


Figura 6.4: Memento

6.4.1 Mapping nel codice

L'implementazione del pattern **Memento** è distribuita tra le classi che gestiscono lo stato dell'appunto e lo stato di persistenza della cronologia:

- **Note**: Svolge il ruolo di *Originator*. Detiene lo stato interno dell'appunto (testo, autore, ID) e implementa i metodi **saveToMemento()** per creare un'istantanea e **restoreFromMemento()** per il ripristino di uno stato precedente.

- **NoteMemento**: Rappresenta l'oggetto *Memento*. Incapsula i dati in modo immutabile e protegge l'incapsulamento fornendo lo stato tramite l'interfaccia stretta `getState()`, accessibile solo dall'Originator o dal Gateway di persistenza.
- **VersionsGateway**: Agisce come *Caretaker*. È l'unico componente responsabile della custodia fisica dei memento, gestendo la loro scrittura nel database (`saveVersion`) e il recupero dal file system.
- **VersionController**: Coordina il flusso operativo, istruendo l'Originatore a generare il memento e delegando al Caretaker la sua conservazione all'interno di una transazione atomica.

6.5 Design Pattern Protection Proxy

Il pattern Protection Proxy si configura come un'architettura di intermediazione in cui un oggetto **Proxy** agisce da surrogato per un **RealSubject**, regolandone l'accesso in modo rigoroso. L'elemento cardine di tale struttura è l'astrazione definita dal **Subject**, che funge da contratto formale tra il sistema e il **Client**. Grazie a questo disaccoppiamento, il **Client** interagisce esclusivamente con l'interfaccia astratta, ignorando se l'esecuzione avvenga tramite l'oggetto reale o il suo sostituto. Questa trasparenza operativa permette al sistema di evolvere e di integrare nuove logiche di sicurezza senza impattare minimamente sulle dipendenze esterne o sul codice chiamante.

Design Pattern: Protection Proxy

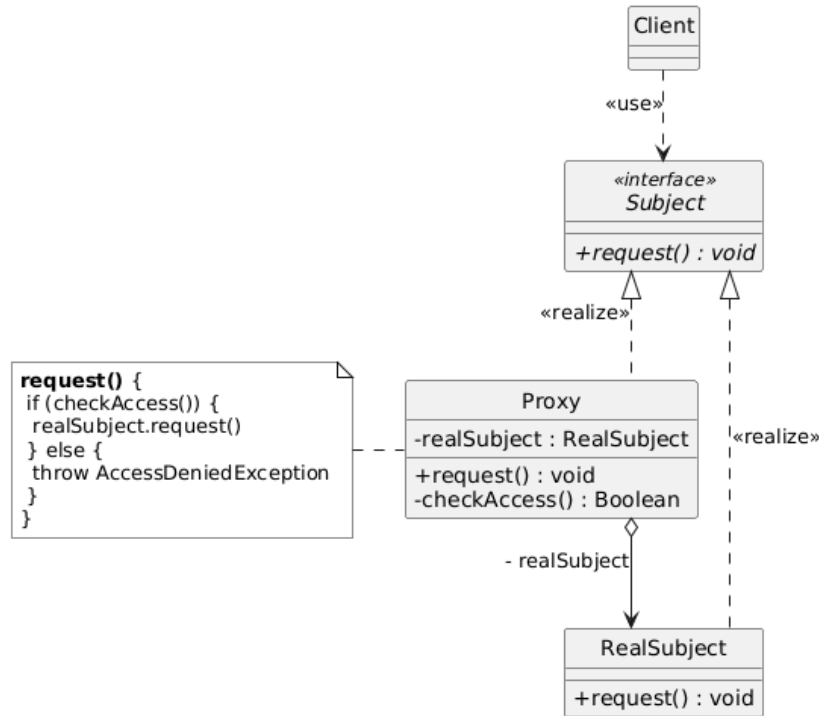


Figura 6.5: Protection Proxy

6.5.1 Mapping nel codice

La logica di sicurezza autoritativa è implementata tramite una serie di **Proxy** che filtrano l'accesso ai componenti di persistenza reale:

- **INotesGateway, IArgomentiGateway, IVersionsGateway, ICoursesGateway**: Definiscono l'interfaccia *Subject*, che funge da contratto formale sia per il proxy che per il gateway reale, garantendo la trasparenza per il client (Controller).
- **NotesGatewayProxy e VersionsGatewayProxy**: Implementano il ruolo di *Proxy* per le risorse dinamiche. Intercettano le chiamate a metodi sensibili (come `createNote` o `saveVersion`), verificano il ruolo studente e negano l'accesso in caso di privilegi insufficienti.
- **NotesGateway, ArgomentiGateway, CoursesGateway e VersionsGateway**: Rappresentano i *RealSubject*. Contengono la logica SQL (PDO) effettiva e la gestione del file system. Vengono invocati dai rispettivi proxy solo dopo il superamento con successo dei controlli autoritativi.

- `CoursesGatewayProxy` e `ArgomentiGatewayProxy`: Specializzazioni del proxy che proteggono la struttura del catalogo, riservando le operazioni di modifica ed eliminazione esclusivamente agli utenti con ruolo amministratore.

Capitolo 7

Piano di Test

7.1 Introduzione alla Validazione

Il presente Piano di Test definisce le procedure necessarie per verificare che il sistema *Wiki Collaborativo* sia conforme alle specifiche e soddisfi i bisogni reali degli utenti (*Verifica & Validazione*). La strategia adottata si basa sul **black-box testing**, in cui i casi di test sono derivati direttamente dai requisiti funzionali (RF) e non funzionali (RNF), testando il sistema tramite input selezionati per coprire sia scenari di successo che condizioni di errore.

7.2 Casi di Test e Tracciabilità

Ogni test case è progettato per garantire la **tracciabilità** verso il relativo requisito, assicurando che ogni funzionalità sia verificabile e correttamente implementata.

ID Test	Requisito	Input / Scenario	Risultato Atteso
T1	RF1	Inserimento di dati validi per un nuovo utente.	Status 200 (Success), nuovo utente generato correttamente.
T2	RF1	Registrazione con una email già presente nel database.	Status 409 (Conflict), messaggio "Email già registrata".
T3	RF2	Inserimento di password errata durante il login.	Status 401 (Unauthorized), accesso negato.
Continua nella prossima pagina			

<i>Continua dalla pagina precedente</i>			
ID Test	Requisito	Input / Scenario	Risultato Atteso
T4	RF2	Inserimento di password corretta durante il login.	Status 200 (Success), accesso autorizzato.
T5	RF6	Accesso alla pagina di un argomento come utente non autenticato (Visitatore).	Il pulsante "+ Crea Nuovo Appunto" non è presente nel DOM.
T6	RF6 & RNF4	Invio manuale di una richiesta POST all'endpoint /appunto/crea senza una sessione valida.	Status 403 (Forbidden), intercettazione del <i>Protection Proxy</i> .
T7	RF10 & RNF4	Accesso con ruolo studente e tentativo di invio manuale di una richiesta DELETE all'endpoint /corso/elimina.	Status 403 (Forbidden), operazione negata dal Proxy.
T8	RNF4	Ispezione persistenza: Verifica del record utente nel DB dopo la registrazione.	Password memorizzate esclusivamente come hash sha256.
T9	RF9	Ricerca del titolo di un appunto con una stringa inferiore a 2 caratteri.	La ricerca non viene avviata dal frontend (ottimizzazione).
T10	RF7	Modifica di un appunto esistente da parte di uno studente autenticato.	Aggiornamento del file .md e creazione di un record in versioni .
T11	RF10 & RNF4	Eliminazione di un argomento da parte di un utente con ruolo amministratore .	Status 200 (Success), pulizia tramite <i>CascadeService</i> .
<i>Continua nella prossima pagina</i>			

<i>Continua dalla pagina precedente</i>			
ID Test	Requisito	Input / Scenario	Risultato Atteso
T12	RF3 & RF4	Navigazione tra i corsi e argomenti.	Visualizzazione corretta della gerarchia nel frontend.
T13	RF5	Apertura di un appunto esistente.	Rendering HTML corretto da sorgente Markdown; metadati coerenti con il DB.
T14	RF8	Visualizzazione delle versioni.	Lista mostrata con timestamp e autore; anteprima disponibile nel modale.
T15	RF8	Ripristino di una versione.	Promozione della versione a corrente; ricaricamento dell'appunto aggiornato.
T16	RF8	Visualizzazione cronologia vuota.	Messaggio: "Nessuna versione precedente disponibile."
T17	RF9	Ricerca di un titolo esistente con stringa valida (es. "Meccanica").	Restituzione dell'array JSON contenente i match corrispondenti.
T18	RF2	Esecuzione Logout.	Distruzione del token/sessione; reindirizzamento alla vista ospite.

Tabella 7.1: Tabella dei Casi di Test: Scenari di Validazione

7.3 Considerazioni sulla Robustezza

Il sistema è stato testato per gestire le anomalie tramite la generazione di eccezioni nello strato dei *Gateway* e dei *Proxy*. In particolare, l'uso di **prepared statements** SQL garantisce la robustezza contro attacchi di *SQL Injection*.

Capitolo 8

Manuale Tecnico

8.1 Introduzione

Il sistema **Wiki UNIPR** è progettato per essere eseguito in un ambiente containerizzato tramite **Docker**. Questa architettura garantisce che il software funzioni in modo identico su qualsiasi macchina, isolando le dipendenze del Backend (PHP), del Database (MySQL) e del Frontend (Apache).

8.2 Requisiti di Sistema

Prima di procedere, assicurarsi che i seguenti strumenti siano installati e correttamente configurati:

- **Docker Desktop** (Windows/macOS) o **Docker Engine** (Linux).
- **Docker Compose** (solitamente incluso nelle versioni recenti di Docker).
- Un terminale (Bash, PowerShell o WSL).

8.3 Inizializzazione del Database

Il database viene configurato automaticamente durante il primo avvio del container DB.

8.3.1 Logica di Popolamento

Docker esegue gli script SQL presenti nella cartella `./services/database/configs/` seguendo l'ordine alfabetico dei nomi dei file. Per questo motivo, i file sono stati nominati con un prefisso numerico:

1. `01_init.sql`: Contiene i comandi `DROP DATABASE IF EXISTS` e `CREATE DATABASE` per garantire un ambiente pulito, seguiti dalla creazione delle tabelle con i relativi vincoli di integrità.
2. `02_dump.sql`: Contiene gli `INSERT` iniziali per popolare la tabella `corsi`, gli argomenti di test e gli utenti predefiniti.

8.4 Script di Automazione

Per semplificare la gestione dell'ambiente, sono stati predisposti tre script `.sh` nella root del progetto.

8.4.1 Avvio del Progetto (`run_project.sh`)

Utilizzare questo script per avviare l'intero stack. Lo script richiede all'utente se desidera effettuare un reset dei volumi per ricaricare il database da zero.

```
./run_project.sh
```

8.4.2 Arresto del Progetto (`stop_project.sh`)

Ferma i container in esecuzione senza eliminare i dati salvati nel database.

```
./stop_project.sh
```

8.4.3 Reset Totale (`reset_all.sh`)

Elimina container, volumi e immagini, ricostruendo l'ambiente da uno stato pulito. Utile in caso di modifiche strutturali al file `init.sql`.

```
./reset_all.sh
```

8.5 Procedura di Installazione Standard

1. Clonare il repository o estrarre i file del progetto.
2. Aprire il terminale nella cartella principale.
3. Fornire i permessi di esecuzione agli script:

```
chmod +x *.sh
```

4. Eseguire lo script di avvio:

```
./run_project.sh
```

5. Attendere circa 10-15 secondi per consentire a MySQL di completare l'inizializzazione.
6. Accedere all'app tramite browser all'indirizzo:

```
http://localhost:8080
```

8.6 Credenziali di Test

Per verificare le funzionalità del sistema (Login, consultazione e permessi), sono stati predisposti i seguenti account nel file di dump:

Tabella 8.1: Credenziali di test pre-caricate nel sistema.

Tipo Utente	Email	Password	Ruolo
Amministratore	admin@unipr.it	Vector!	amministratore
Studente	studente_test1@studenti.unipr.it	Prova1	studente
Studente	studente_test2@studenti.unipr.it	Prova2	studente
Studente	studente_test3@studenti.unipr.it	Prova3	studente

8.7 Risoluzione dei Problemi

Se le tabelle non dovessero risultare presenti (*Table not found*), eseguire il comando `./reset_all.sh`. Questo forzerà Docker a cancellare il volume dati esistente e a rieseguire gli script di inizializzazione `01_init.sql` e `02_dump.sql`.